

PROJECT REPORT

TO-DO LIST MANAGEMENT SYSTEM

A Python-Based Task Management Project

Submitted By: Gayatri Yadav

Course: B.Tech – CSE (Cloud Computing and Automation)

Semester: 1st Semester

Subject: Python Programming

Institute: VIT Bhopal University

Academic Year: 2026–27

Acknowledgement

I would like to express my sincere gratitude to my Python Programming faculty for giving me the opportunity to work on this project. This project helped me understand how basic Python concepts can be combined to solve a simple real-life problem.

I am also thankful to my classmates, friends, and learning resources that supported me while developing and testing the project. Working on this project gave me practical experience in writing, testing, and improving a Python program.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Project Overview

5. Features of the Project
6. Technologies and Tools Used
7. Python Concepts Used
8. Working of the System
9. Algorithm / Program Flow
10. Sample Output
11. Testing
12. Advantages
13. Limitations
14. Future Enhancements
15. Conclusion
16. References

1. Introduction

The To-Do List Management System is a simple command-line application developed using Python. It is designed to help a user manage daily tasks in an organized way. The user can add tasks, view them, mark completed tasks, delete tasks, and check overall progress.

The project was created as a practical application of basic Python concepts. It focuses on a simple menu-driven interface while solving a useful everyday problem.

2. Problem Statement

When a person has several assignments, study activities, or personal tasks, it can become difficult to keep track of everything. A simple task management system can help record tasks and show which tasks are completed and which are still pending.

The aim of this project is to develop a small Python program that provides these basic task-management functions through a simple menu.

3. Objectives

- To develop a simple and easy-to-use task management application.
- To practice Python lists, dictionaries, loops, and conditional statements.
- To learn how to take and validate user input.

- To allow users to add, view, complete, and delete tasks.
- To calculate and display task completion progress.
- To handle invalid input using exception handling.
- To apply programming concepts to a practical problem.

4. Project Overview

The application works through a menu displayed in the terminal. The user selects an option by entering a number. The selected operation is performed and the program returns to the main menu until Exit is selected.

===== MY TO-DO LIST =====

1. Add Task
2. View Tasks
3. Mark Task as Completed
4. Delete Task
5. Progress Check
6. Exit

5. Features of the Project

5.1 Add Task

The user can enter a new task and add it to the task list. Its initial status is Pending.

5.2 View Tasks

The program displays the stored tasks with their task numbers and current status.

5.3 Mark Task as Completed

The user selects a task number and changes its status from Pending to Completed.

5.4 Delete Task

The user selects a task number and removes that task from the list.

5.5 Progress Check

The program calculates the total, completed, and pending tasks and displays the completion percentage.

===== PROGRESS CHECK =====

Total tasks: 4

Completed tasks: 3

Pending tasks: 1

Completion percentage: 75.0%

5.6 Input Validation

The program checks task numbers and uses try-except where required so invalid input does not stop the program unexpectedly.

6. Technologies and Tools Used

Technology / Tool	purpose
Python	Main programming language
Visual Studio Code	Writing, editing, and running the program
Git	Version control
GitHub	Storing and sharing the project

7. Python Concepts Used

Lists: Store multiple tasks.

Dictionaries: Store task details such as name and completion status.

Variables: Store choices, counts, and progress values.

Input and Output: Take user input and display results.

Conditional Statements: Choose operations using if, elif, and else.

Loops: Keep the main menu running.

Functions: Can divide operations into reusable blocks.

Exception Handling: Handle invalid numerical input using try-except.

8. Working of the System

- The program starts and creates the task list.
- The main menu is displayed.
- The user selects an option.
- The selected operation is performed.
- The program validates task numbers where necessary.
- The menu appears again after the operation.
- The loop ends when Exit is selected.

9. Algorithm / Program Flow

START

↓

Display Main Menu

↓

Take User Choice

↓

Add / View / Complete / Delete / Progress

↓

Return to Main Menu

↓

Exit selected?

↓

YES → STOP

NO → Display Menu again

10. Sample Output

10.1 Adding a Task

Enter your choice (1-6): 1

Enter your task: Complete Python Assignment

Task added successfully!

10.2 Viewing Tasks

===== YOUR TASKS =====

1. Complete Python Assignment - Pending
2. Complete EVS Notes - Completed

10.3 Completing a Task

Enter the task number to mark as completed: 1

Task marked as completed!

10.4 Progress Check

===== PROGRESS CHECK =====

Total tasks: 2

Completed tasks: 1

Pending tasks: 1

Completion percentage: 50.0%

11. Testing

The program was tested with different inputs to check that its main features work correctly.

Test Case	Action	Expected Result
Add task	Select 1 and enter a task	Task is added
View tasks	Select 2	Tasks are displayed
Complete task	Select 3 and valid number	Status changes to Completed
Delete task	Select 4 and valid number	Task is removed
Progress	Select 5	Statistics are displayed

Invalid input	Enter invalid number	Error message is displayed
Exit	Select 6	Program terminates

12. Advantages

- Simple and easy to use.
- Helps organize daily tasks.
- Demonstrates basic Python concepts in a practical way.
- Shows completed and pending work clearly.
- Provides a quick progress summary.
- Easy to extend with more features.

13. Limitations

- The current version uses a command-line interface.
- Task data is temporary if permanent storage is not implemented.
- It is designed for basic single-user task management.
- It does not provide automatic notifications or reminders.

14. Future Enhancements

- Add High, Medium, and Low priority levels.
- Add due dates and deadline reminders.
- Add search and filtering.
- Store tasks permanently using JSON or a database.
- Add task categories.
- Develop a graphical user interface.
- Add notifications and reminders.

15. Conclusion

The To-Do List Management System is a small but practical Python project. It helped me understand how different programming concepts can be combined to solve a simple real-life problem.

During development, I gained practical experience with lists, dictionaries, loops, conditional statements, user input, and exception handling. I also learned the importance of testing a program with different inputs and improving it step by step.

The project can be extended with priorities, deadlines, permanent storage, search, and a graphical interface. Overall, it provided a useful foundation for learning Python and developing larger applications.

16. References

- Python class notes and lecture material.
- Python documentation and basic programming references.
- Practice programs developed during Python learning